



Unit 3 – Software Design

Object Oriented Design

Learning Outcomes



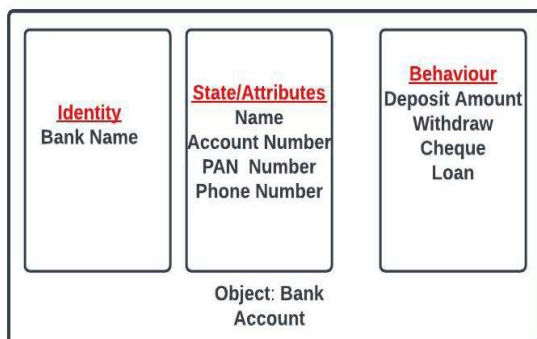
- **Understand object-oriented design (OOD)**

OOD

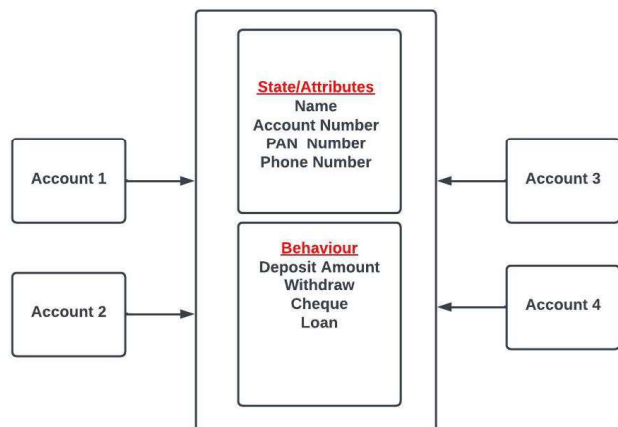
- OOD is a conceptual model where a system is viewed as a collection of objects.
- Object is an entity in a solution. For Example offices, users, schools, shops, companies, etc.
- Object is described using class. A class is an abstract data type that involves attributes and methods.
- Object is an instance of a class.

Objects and Classes

(1) Object for Bank Account



(2) A Single class with many objects



OOD Concepts

- **Abstraction highlights important properties and removes an unimportant or irrelevant information**
- **Encapsulation combines data and functions together with data hiding**
- **Inheritance allows reusability**
- **Polymorphism allows the data to be processed in more than one form**

OOD Design Process

- **Define the context of the use of the system**
- **Design the system architecture**
- **Identify the important/principal system objects**
- **Develop design models**
- **Specify object interfaces**

Unit 3 – Software Design

Unified Modelling Language (UML)

Learning Outcomes

- **Understand unified modelling language (UML)**

UML

- Many distinct notations for describing the OOD process were proposed in the 1980s and 1990s.
- The **UML** is an integration of these notations that describes different design models that are developed during OO analysis and design.
- It is now a de facto standard for OO modeling.
- The notations in **UML** are divided into three categories: **Things**, **relationships**, and **diagrams**.

UML Concepts

- **Things**: class, object, interface, use-case, actor, package, components
- **Relationships**: Start and stop state, action box, decision box, annotation (explanation), etc.
- The relationships in UML are **Association**, **Dependency**, **Generalization**, **Realization**
- The different diagrams are used to show the **structure**, **behavior**, and **interaction** in a system

UML in OOD

- **Structural diagrams:** **Class diagrams**, which show the object classes in the system and their associations.
- **Interaction diagrams:** These diagrams show interaction, for example, **use case diagrams**, show the interactions between a system and its environment, and **sequence diagrams** show interactions between actors, the system, and system components.
- **Behavioural diagrams:** **Activity diagrams** show the activities involved in a process or in data

Unit 3 – Software Design

Structured Models

Learning Outcomes

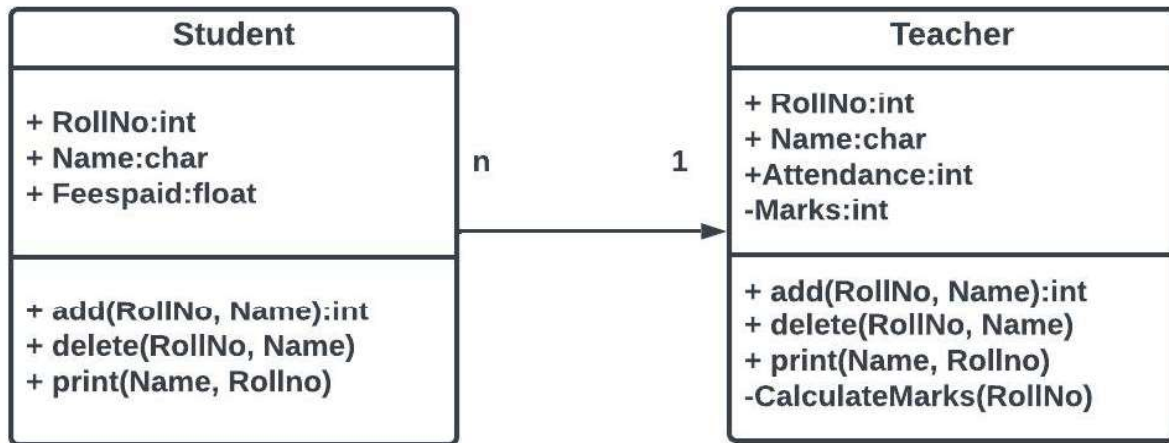
- **Understand the structured models**
- **Appreciate the use of class diagrams**
- **Understand relationships in UML**

Structured Models

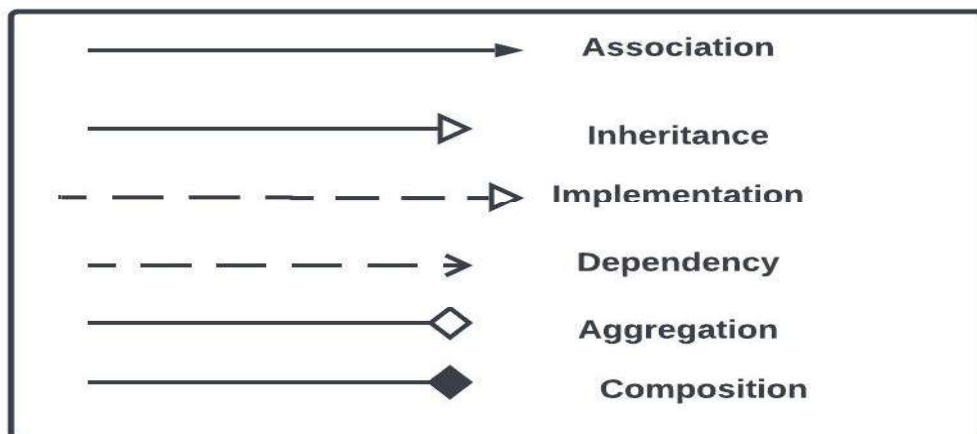
- **Structured models display the organization of a system**
- **Structural models may be static or dynamic**
- **Static models represent the structure of a system design**
- **Dynamic models show the system during execution**
- **Example: class diagram**

Class Diagrams

Class Name, attributes, operations or methods, and the relationships among objects



UML Relationships





Unit 3 – Software Design

Context and Interaction Models

Learning Outcomes



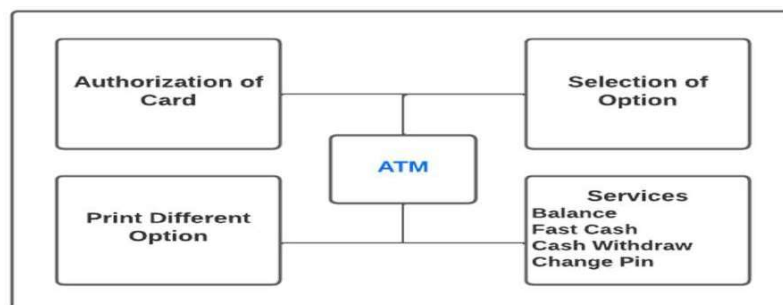
- **Understand context models in OOD**
- **Understand the interaction models in OOD**

Models

Models are designed from three perspectives

- External perspective
- Interaction perspective
- Behavioural perspective

Context models



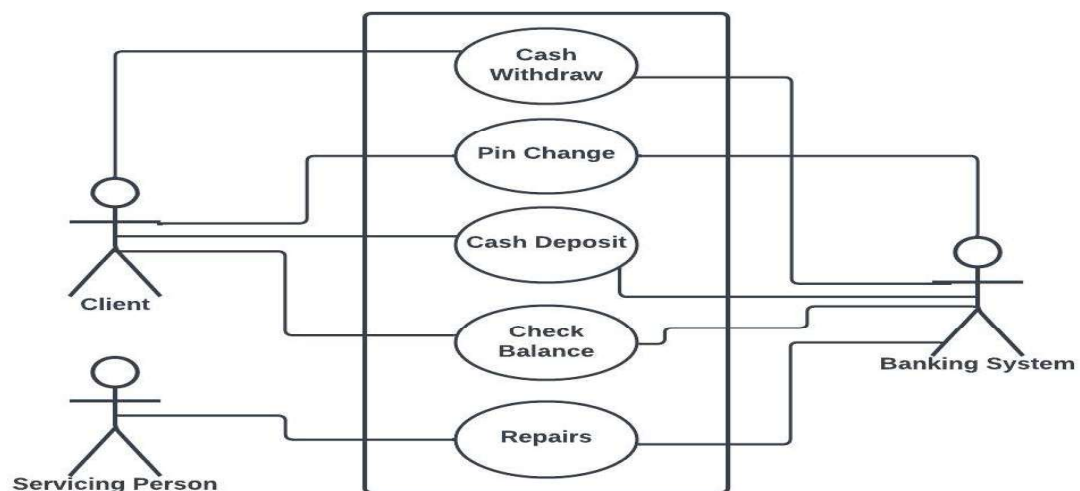
- The context models are used to show the scope, different functionalities in the system and beyond the system's boundary.
- Context models are very starting point of the design phase of a software.

Interaction Models

- **Use case modeling or Use case diagrams:** The interaction of how the system works with actors and test cases are shown.
- A use case is shown as an ellipse with the actors involved in the use case represented as stick figures



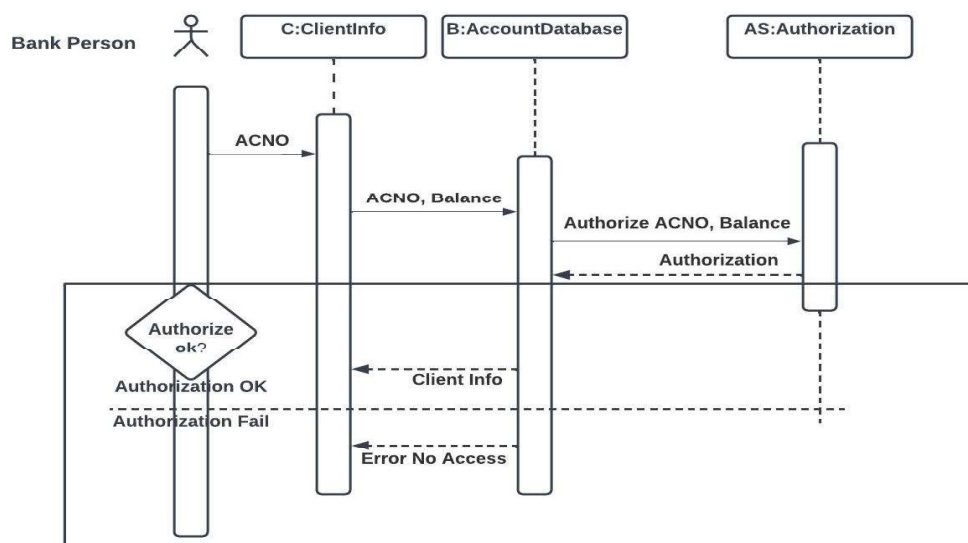
Use Case Diagrams



Sequence Diagrams

- Model the interactions between objects in a single-use case
- Shows different parts of a system work in a 'sequence' to get something done.
- Symbols: Lifetime Notations, Actors, Activation bars

Sequence Diagrams

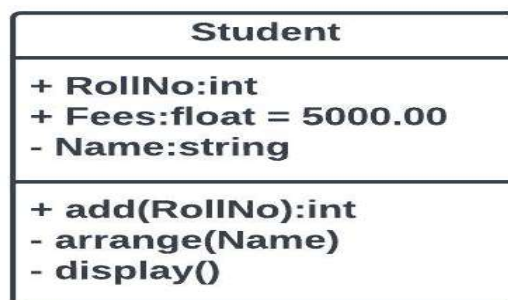


Unit 3 – Software Design

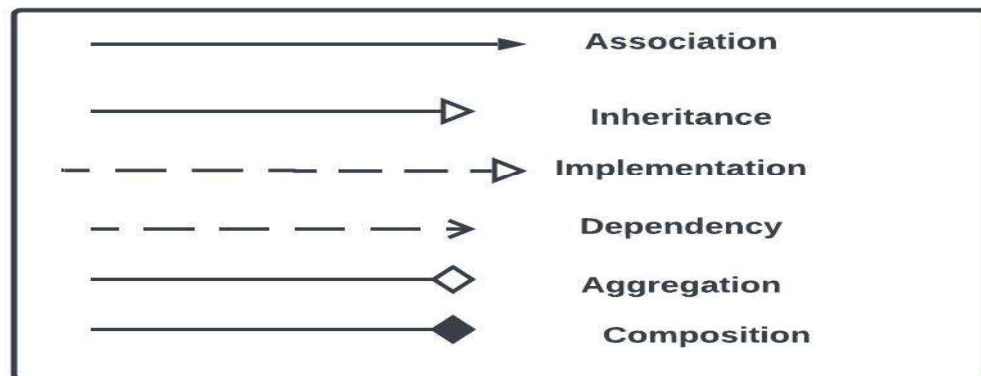
OOD Relationships: Examples

Class Diagram

- Class diagrams gives overview of a software with attributes, classes, functions, and relationships

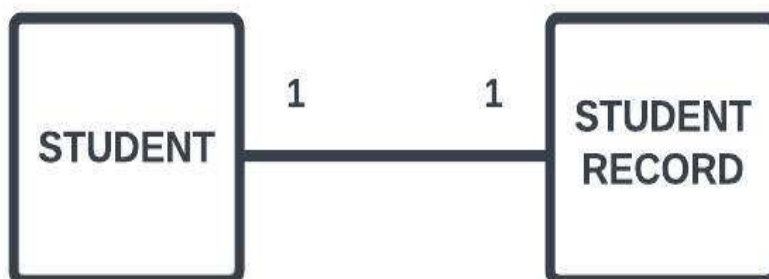


Relationships

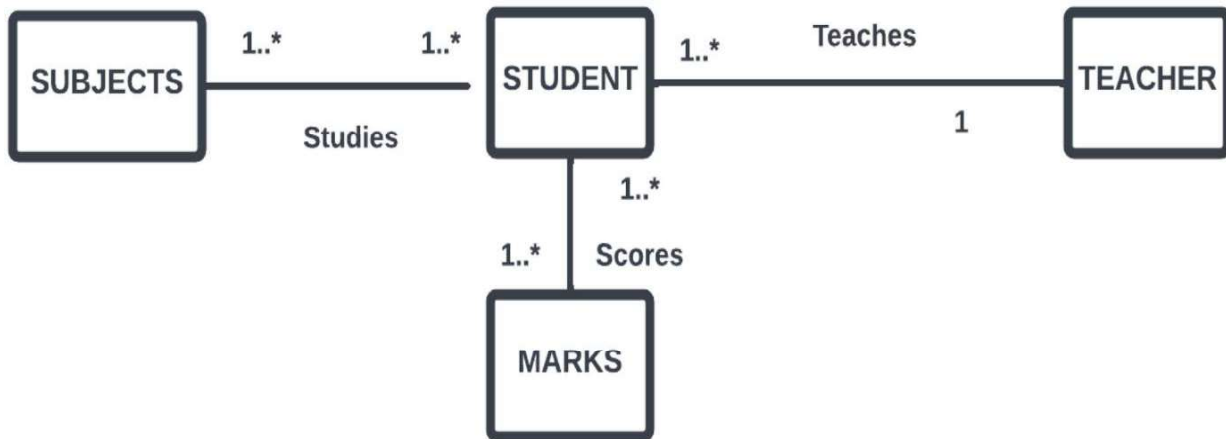


Association

- Association: Known as “*has a*” relationship.
- A student *has a* record, a bus *has an* engine and wheels

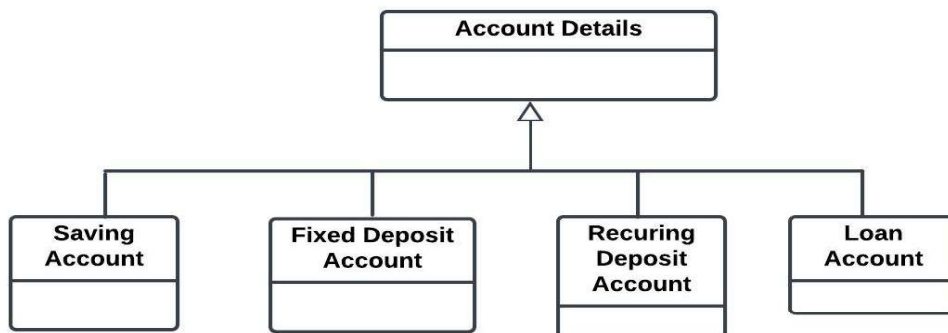


Association

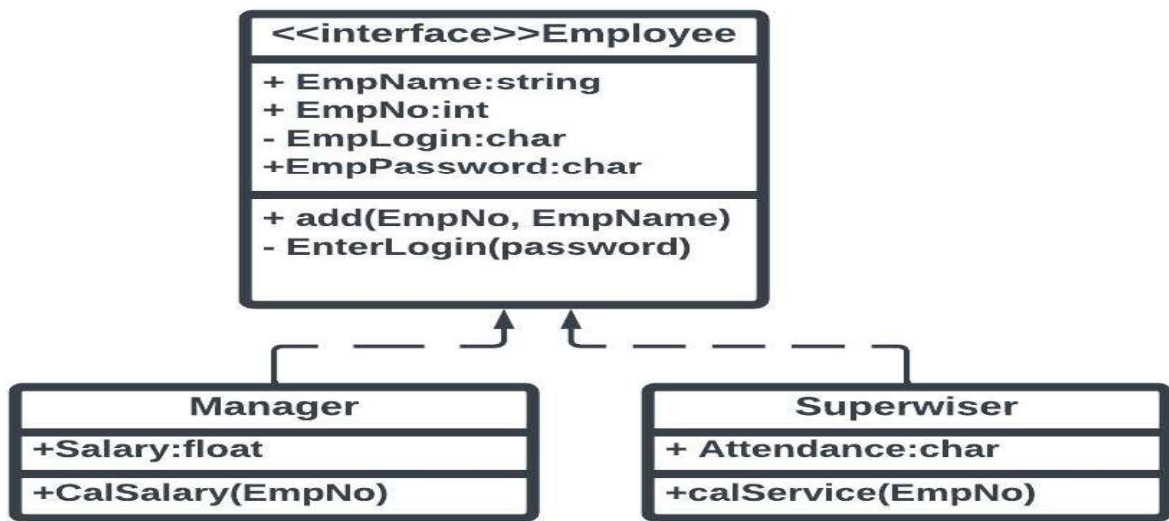


Generalization/Inheritance

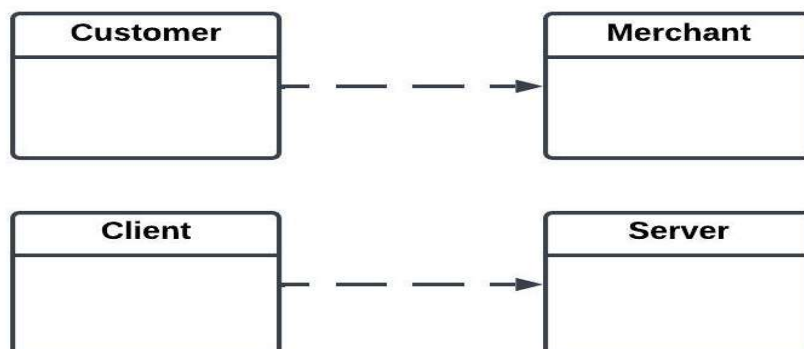
- Combining similar objects in a generic class, Also known as Inheritance.



Implementation/Realization

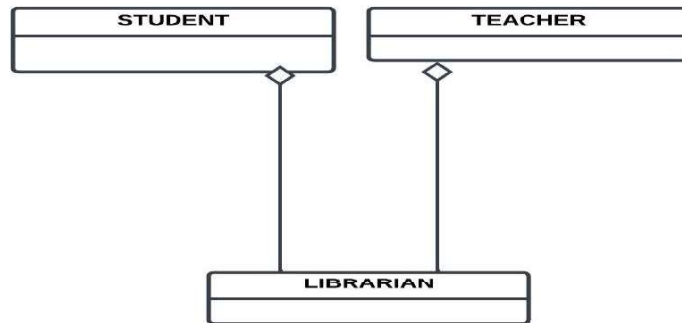


Dependency



Aggregation

- It describes a **part-whole** or **part-of** relationship. Here the child can exist independently of the parent.



Composition

- Composition implies one class containing another. The contained class cannot exist independently
- A library cannot exist without Books





Unit 3 – Software Design

Behavioural Diagrams

Learning Outcomes

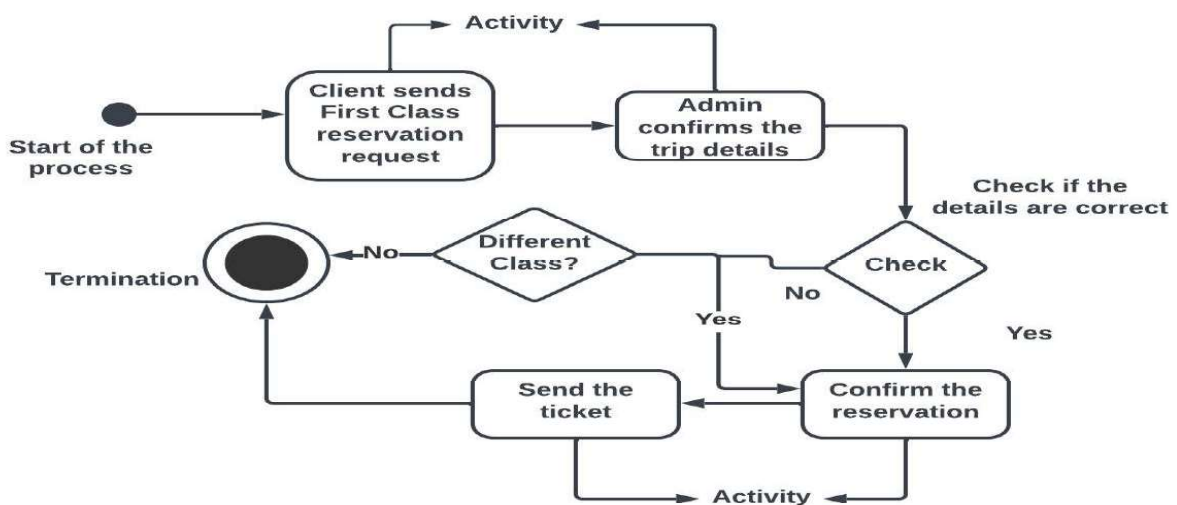


- Understand activity diagrams

Data and Event Driven Modelling

- Behavioural models show the system in execution. It is either data or event-driven.
- For example, a telephone bill is generated using calls and the internet usage of a customer. This is an example of the system responding to data or Data driven modelling
- When a landline telephone switching office responds to “dial a number”, “keep the receiver on the hook”. This is an example of event-driven modelling

Activity Diagram-Data Driven





Unit 3 – Software Design

Behavioural Diagrams

Learning Outcomes

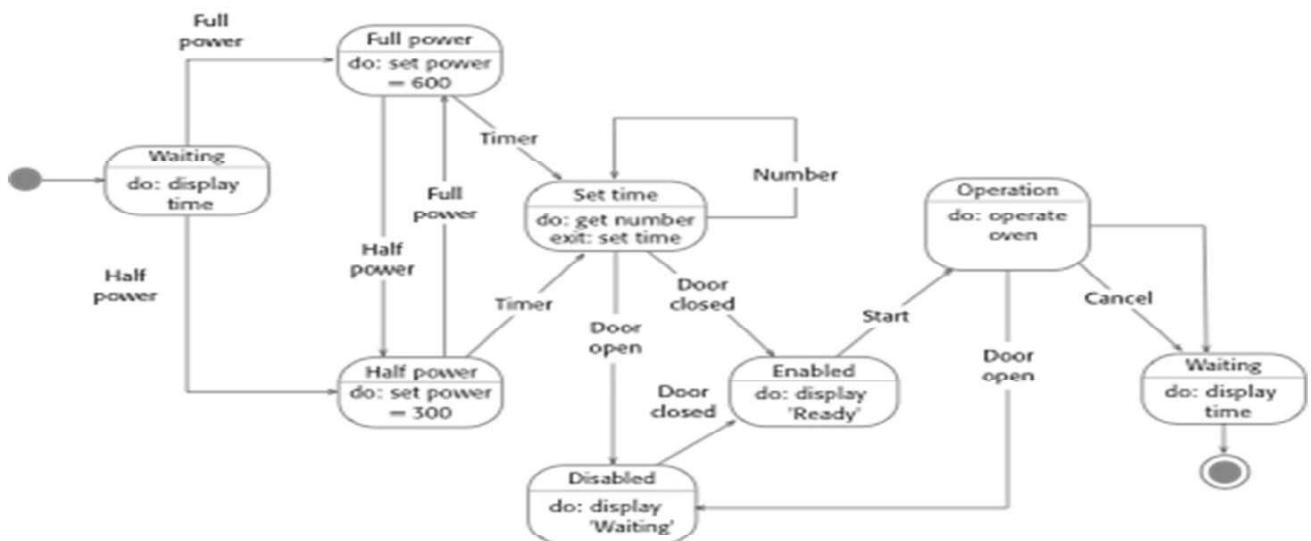


- **Understand state diagrams**

State Diagram-Event Driven

- Event is termed as **stimuli**
- Event-driven modeling is the response of a system to external and internal events
- State diagrams are **event-driven**
- They show a behavioral model using states, state transitions, and actions

State Diagram-Microwave Oven



States and Stimuli

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

States and Stimuli

Stimulus	Description
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.

References

- Sommerville, Ian. Software Engineering. 9 Harlow, England: Addison-Wesley, 2010.

Unit 3 – Software Design

Architectural Design

Learning Outcomes

- Understand the importance of architectural design
- Understand the different views of software architecture
- Identify different patterns for architectural design

Architectural Design

- It serves as a blueprint of a system
- It describes the organization and design of the
the overall structure of a system
- It is the starting stage in the software design process
- It is the important link between the requirements and design

Advantages

Architectural design is related to many **non-functional requirements** such as **performance, safety, security, availability, and maintainability**

- **Stakeholder communication**
- **System analysis**
- **Large-scale reuse**

Architectural Views

- **A logical view**
- **A process view**
- **A development view**
- **A physical view**

Unit 3 – Software Design

Architectural Design Pattern

Architectural Design Pattern

- The architecture of a software system may be based on a particular architectural pattern or style
- **Architectural patterns** give an abstract description of a system organization
- It is helpful in presenting, sharing, and reusing knowledge about software systems

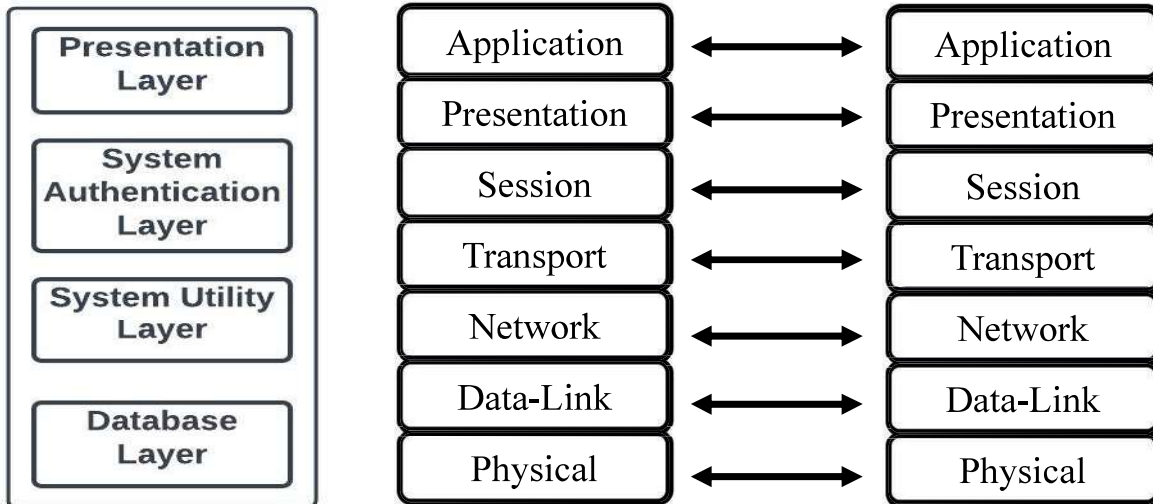
Architectural Design Pattern

- Layered Architecture
- Repository Architecture
- Client-Server Architecture
- Pipe-filter Architecture

Layered Architecture

Description	Organization of the system is into layers, with related functions associated
Example	OSI Network model. Gmail layered model
When Used	Distributed functionality, when new services are built over existing system
Advantages	Layered approach is simple and easy to implement; Less dependency; Testing and maintenance is easier because of separate layered functions
Disadvantages	Clear separation between layers, scalability and parallel processing is difficult in layered approach

Example



Unit 3 – Software Design

Repository Architecture

Repository Architecture

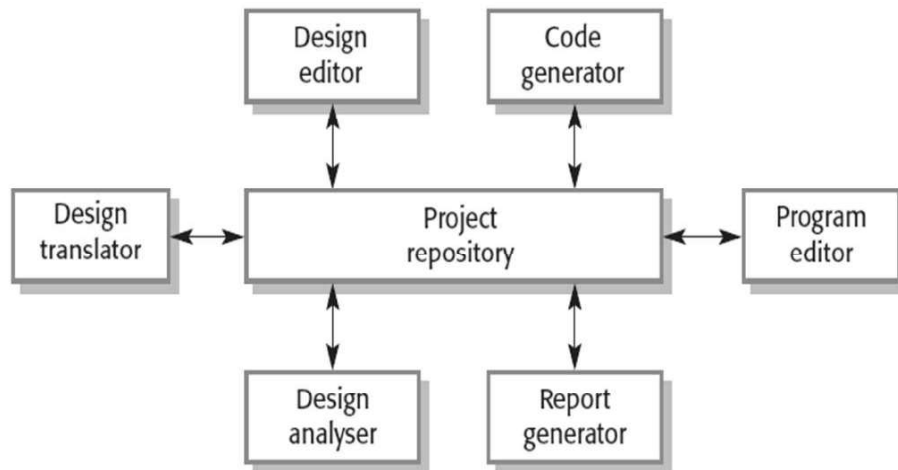
- Large amounts of data are organized around a shared database or repository
- Examples: Command and control systems, management information systems, Computer-Aided Design (CAD) systems, and interactive development environments for software
- Sub-systems must exchange data so that they can work together effectively. This may be done with a single repository or with each subsystem maintaining database

Repository Architecture

Description	All data in a system is saved in a central repository. It is accessible to all system components. Components interact only through the repository
Example	IDE, CASE Tools
When Used	System with voluminous data with long life and in data-driven systems
Advantages	Centralized data and other activities such as backup, security, recovery etc.
Disadvantages	The failure of the single point used as a repository may shut down the whole system. The distribution and evolution is difficult

Example

□ CASE: Computer Aided Software Engineering



Unit 3 – Software Design

Client-Server Architecture Design

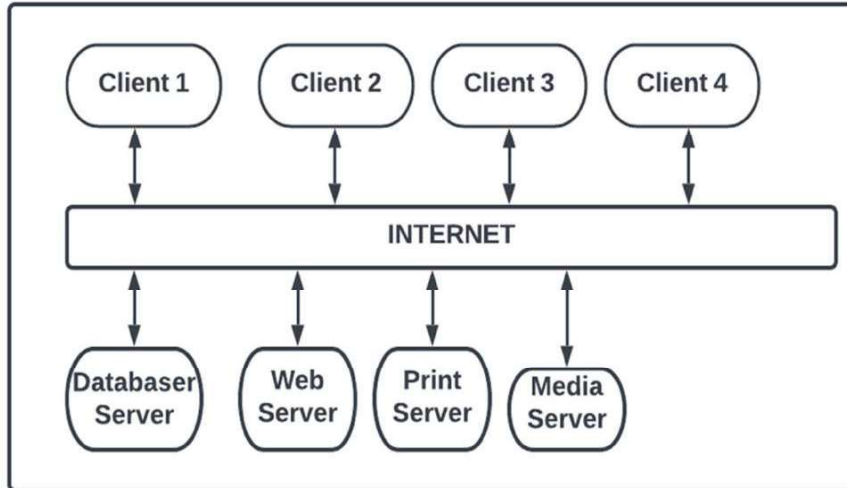
Client-Server Architecture

- **Distributed architecture where the functions in the system are performed using clients and servers.**
- **The client is a service requester. The client requests the specific service to the server using the internet and the server response.**
- **Example: email, world wide web, etc.**

Client-Server Architecture

Description	The description is information of a set of services and servers. Clients are users of these services and access servers to make use of them.
Example	A network with a client-server system
When Used	During shared database and functions. As servers can be replicated, it is also used when the load on the system is dynamic
Advantages	The client need not have all the functions; servers can be distributed across the network.
Disadvantages	Denial of service due to security attacks or server failures. Poor performance due to dependency

Example



Unit 3 – Software Design

Pipe and Filter Architecture

Pipe and Filter Architecture

- The pipe and filter architecture is inspired by the Unix operating system
- This model shows the execution or runtime organization of a system
- Here the filter is a component that transforms the data and processes it as input
- The pipe connects different streams of data flowing in a sequence

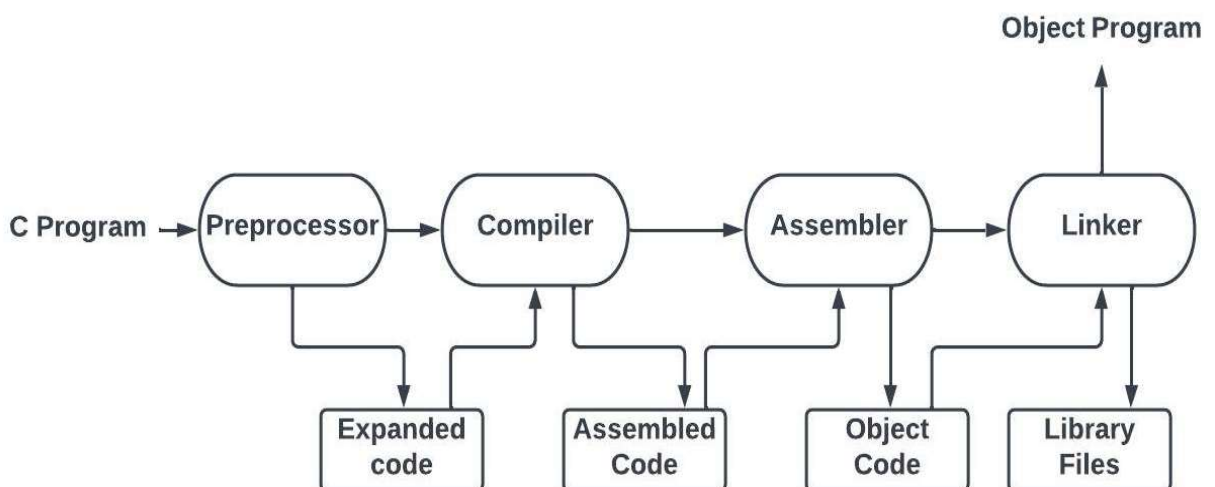
Pipe and Filter Architecture

- Each processing step is implemented as a transform
- Input data flows through these transforms until converted to output
- The transformations may execute sequentially or in parallel
- The data can be processed by each transform item by item or in a single batch

Pipe and Filter Architecture

Description	Filters are discrete and perform data transformation from one component to another for processing
Example	Steps in compilation of a program
When Used	Data-processing applications The data processing may be batch processed or transaction-based
Advantages	Many businesses processes match this approach, which makes it suitable and easy to implement. Easy to understand and supports transformation reuse. It can be used in both sequential or concurrent operations
Disadvantages	The order of data transfer should be decided in advance

Example



Example

